

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені Тараса Шевченка
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
Кафедра програмних систем і технологій

Дисципліна
« Комп'ютерна графіка і візуалізація »

Лабораторна робота № 1

Виконав:	Дацишин Володимир	Перевірила:	Жабська Єлизавета Олегівна
Група	ІПЗ-23		
Форма навчання	денна	Оцінка	
Спеціальність	121		
2026			

Тема: розробка програми для обробки зображень: конвертація форматів, зміна розміру, перетворення кольорів та корекція колірного балансу.

Мета: ознайомитися з основними принципами обробки зображень за допомогою програмного забезпечення; навчитися реалізовувати програму для конвертації форматів зображень, зміни розміру, перетворення кольорів та здійснення корекції колірного балансу; дослідити вплив різних операцій на розмір і якість зображень, а також розвинути практичні навички роботи з графічними бібліотеками для реалізації зазначених функцій.

Хід виконання роботи:

Завдання 1. Конвертація форматів зображення

Хід виконання: Реалізовано програмний модуль (функція *convert_format*), який зчитує вхідні графічні файли та виконує їх експорт у задані формати (PNG, JPEG, WEBP, BMP). Особливу увагу приділено обробці прозорості: при збереженні у формати, що не підтримують альфа-канал, зображення попередньо зводиться з білим фоном для запобігання появі чорних зон. Програма також проводить порівняльний аналіз розміру файлів (у КБ) до та після конвертації.

Використані функції та методи:

- `os.path.isfile()`, `os.path.getsize()`, `os.path.splitext()` — для базових операцій з файловою системою (перевірка наявності, отримання розміру, розбір розширення).
- `Image.open()` (з бібліотеки Pillow) — для ініціалізації об'єкта зображення.
- `img.convert('RGB')` / `img.convert('RGBA')` — для зміни колірного простору.
- `img.split()` — для сепарації каналів (отримання альфа-маски).
- `Image.new()` та `bg.paste()` — для генерації фонового шару та накладання на нього напівпрозорого зображення.
- `img.save()` — для серіалізації (збереження) результату у файл із вказаним форматом.

Лістинг коду з коментарями:

```
def convert_formats(file_paths: list[str], formats:
list[str], output_dir: str) -> None:
    """Конвертує зображення у задані формати та порівнює
    розміри."""
    os.makedirs(output_dir, exist_ok=True)
    print(f"\n{'Файл':<20} | {'Формат':<7} | {'До
    (КБ)':<10} | {'Після (КБ)':<10} | Різниця")
    print("-" * 65)

    for path in file_paths:
        if not os.path.isfile(path):
            continue
        try:
            with Image.open(path) as img:
```

```

        base_name: str =
os.path.splitext(os.path.basename(path))[0]
        size_before: float = get_size_kb(path)
        for fmt in formats:
            fmt = fmt.strip().lstrip(".").lower()
            save_img: Image.Image = img.copy()
            if fmt in ("jpg", "jpeg", "bmp") and
save_img.mode in ("RGBA", "LA", "P"):
                save_img = save_img.convert("RGBA")
                bg: Image.Image = Image.new("RGB",
save_img.size, (255, 255, 255))
                bg.paste(save_img,
mask=save_img.split()[3])
                save_img = bg
            elif fmt in ("jpg", "jpeg") and
save_img.mode != "RGB":
                save_img = save_img.convert("RGB")

        out_path: str =
os.path.join(output_dir, f"{base_name}.{fmt}")
        save_img.save(out_path)
        size_after: float =
get_size_kb(out_path)
        diff: float = size_after - size_before
        print(f"{base_name[:20]:<20} |
{fmt.upper():<7} | {size_before:<10.2f} |
{size_after:<10.2f} | {diff:+.2f} KB")
    except Exception as e:
        print(f"Помилка конвертації '{path}': {e}")
print("-" * 65)

```

Результат виконання програми:

```

== Лабораторна робота №1: Комп'ютерна графіка ==
1. Конвертація форматів (з порівнянням розмірів)
2. Зміна розміру зображень (пропорційна, точна)
3. Перетворення кольорів (заміна цільового кольору)
4. Корекція колірного балансу та спектр
5. Вихід
Оберіть дію (1-5): 1
Шляхи через кому: test_images/test_landscape.jpg, test_images/test_transparent.png
Формати (png, jpg, webp, bmp): png, webp, bmp
Директорія збереження: output_formats

```

Файл	Формат	До (KB)	Після (KB)	Різниця
test_landscape	PNG	39.75	94.88	+55.13 KB
test_landscape	WEBP	39.75	10.20	-29.55 KB
test_landscape	BMP	39.75	2812.55	+2772.80 KB
test_transparent	PNG	2.64	2.64	+0.00 KB
test_transparent	WEBP	2.64	3.73	+1.09 KB
test_transparent	BMP	2.64	468.80	+466.17 KB

Хід виконання: Розроблено функцію *resize_images*, яка дозволяє динамічно перераховувати габарити зображення. Алгоритм підтримує точне задання ширини/висоти, пропорційне масштабування за однією зі сторін, а також відсоткове збільшення чи зменшення. Для усунення артефактів (аліасингу) застосовуються різні методи передискретизації пікселів. *Використані функції та методи:*

- Атрибут `img.size` — для отримання початкової матриці розмірів (ширина, висота).
- Функції `max()`, `round()`, `int()` — для математичного розрахунку нових пропорцій без виходу за межі допустимих значень (мінімум 1 піксель).
- `img.resize(size, resample)` — метод бібліотеки Pillow, який виконує фізичну зміну розміру.
- Константи `Image.Resampling.LANCZOS`, `BICUBIC`, `BILINEAR`, `NEAREST` — для керування математичним апаратом інтерполяції.

Лістинг коду з коментарями:

```
def resize_images(
    file_paths: list[str], output_dir: str, target_w: int
    | None = None, target_h: int | None = None,
    filter_choice: str = "1"
) -> None:
    """Зміна розміру із вибором інтерполяції та
    збереженням пропорцій."""
    os.makedirs(output_dir, exist_ok=True)
    resample_filter, filter_name =
    RESAMPLE_METHODS.get(filter_choice,
    (Image.Resampling.LANCZOS, "Lanczos"))
    print(f"\nЗастосовано фільтр: {filter_name}")
    print(f"{'файл':<20} | {'Було':<11} | {'Стало':<11} |
    {'До (КВ)':<10} | Після (КВ)")
    print("-" * 65)

    for path in file_paths:
        if not os.path.isfile(path):
            continue
        try:
            with Image.open(path) as img:
                w, h = img.size
                if target_w and target_h:
                    nw, nh = target_w, target_h
                elif target_w:
                    nw, nh = target_w, int(round(h *
target_w / w))
                elif target_h:
                    nw, nh = int(round(w * target_h / h)),
target_h
```

```

        else:
            return

        resized: Image.Image = img.resize((nw,
nh), resample=resample_filter)
        base_name, ext =
os.path.splitext(os.path.basename(path))
        out_path: str = os.path.join(output_dir,
f"{base_name}_{nw}x{nh}{ext}")
        resized.save(out_path)
        print(f"{base_name[:20]:<20} |
{f'{w}x{h}':<11} | {f'{nw}x{nh}':<11} |
{get_size_kb(path):<10.2f} |
{get_size_kb(out_path):.2f}")
        except Exception as e:
            print(f"Помилка зміни розміру '{path}': {e}")
    print("-" * 65)

```

Результат виконання програми:

```

=== Лабораторна робота №1: Комп'ютерна графіка ===
1. Конвертація форматів (з порівнянням розмірів)
2. Зміна розміру зображень (пропорційна, точна)
3. Перетворення кольорів (заміна цільового кольору)
4. Корекція колірного балансу та спектр
5. Вихід
Оберіть дію (1-5): 2
Шляхи через кому: test_images/test_landscape.jpg
Директорія збереження: output_resized
Режим (а. Точні WxH / б. Лише W / с. Лише H): б
Ширина: 600
Фільтр (1.Lanczos 2.Bicubic 3.Bilinear 4.Nearest, дефолт 1): 1

Застосовано фільтр: Lanczos
Файл          | Було          | Стало          | До (КВ)       | Після (КВ)
-----
test_landscape | 1200x800      | 600x400        | 39.75         | 9.85
-----

```

Завдання 3. Перетворення кольорів (заміна заданого кольору)

Хід виконання: Функція `replace_color` виконує пошук пікселів певного кольору та їх перефарбовування. Оскільки чистий колір зустрічається рідко, реалізовано підтримку «толерантності» (похибки) — замінюються всі пікселі, що входять у заданий радіус відтінків. Для обробки зображень з високою роздільною здатністю алгоритм використовує векторні обчислення на базі матриць, що суттєво зменшує час виконання. *Використані функції та методи:*

- Векторний підхід (бібліотека NumPy):
 - `np.array()` — для трансформації зображення у числовий тензор.
 - `np.abs()` — для знаходження модулів різниці між поточним та цільовим кольорами по каналах.
 - `np.all()` — для логічного об'єднання умов створення маски заміни.
 - `Image.fromarray()` — для збирання готового зображення з оновленої матриці.
- Ітеративний підхід (Pillow):
 - `img.getdata()` та `img.putdata()` — для прямого читання та запису списку пікселів у разі неможливості використання NumPy.
- Вбудована функція `int(hex_str, 16)` — для декодування HEX-формату кольорів.

Лістинг коду з коментарями:

```
def replace_color(
    input_path: str, output_path: str, target_rgb:
    tuple[int, int, int], new_rgb: tuple[int, int, int],
    tolerance: int = 0
) -> bool:
    """Заміна цільового кольору на новий з урахуванням
    похибки tolerance."""
    if not os.path.isfile(input_path):
        return False
    os.makedirs(os.path.dirname(output_path) or ".",
    exist_ok=True)
    with Image.open(input_path) as img:
        work_img: Image.Image = img.convert("RGBA" if
        img.mode == "RGBA" else "RGB")
        arr: np.ndarray = np.array(work_img)
        diff: np.ndarray = np.abs(arr[:, :,
        :3].astype(np.int32) - np.array(target_rgb,
        dtype=np.int32))
        mask: np.ndarray = np.all(diff <= tolerance,
        axis=-1)
        replaced: int = int(np.count_nonzero(mask))
        arr[mask, :3] = new_rgb

        result: Image.Image = Image.fromarray(arr)
```

```

        if output_path.lower().endswith((".jpg", ".jpeg"))
and result.mode != "RGB":
            result = result.convert("RGB")
            result.save(output_path)

        total: int = img.width * img.height
        pct: float = (replaced / total * 100) if total > 0
else 0.0
        print(f"Збережено: {output_path} | Замінено:
{replaced:,} з {total:,} пікселів ({pct:.2f}%)")

```

Результат виконання програми:

```

=== Лабораторна робота №1: Комп'ютерна графіка ===
1. Конвертація форматів (з порівнянням розмірів)
2. Зміна розміру зображень (пропорційна, точна)
3. Перетворення кольорів (заміна цільового кольору)
4. Корекція колірнього балансу та спектр
5. Вихід
Оберіть дію (1-5): 3
Вхідний файл: test_images/test_shapes.png
Вихідний файл: output_colors/shapes_replaced.png
Колір для заміни (назва, HEX або R,G,B): red
Новий колір: black
Похибка (0-255, дефолт 0): 5
Збережено: output_colors/shapes_replaced.png | Замінено: 38,025 з 480,000 пікселів (7.92%)

```

Завдання 4. Корекція колірнього балансу та аналіз спектра

Хід виконання: Завдання реалізоване комплексно через серію функцій.

adjust_color_balance виконує ручне лінійне зміщення значень каналів R, G, B.

auto_color_balance проводить автоматичну нормалізацію (приведення середньої яскравості каналів до нейтрального значення 128).

analyze_color_balance

розраховує відносну частку кожного каналу (у відсотках) для виявлення

колірних перекосів. *extract_color_channels* формує три окремі файли, залишаючи на кожному лише один активний колірний спектр. Використані функції та

методи:

- `img.split()` — для декомпозиції зображення на окремі червоний, зелений та синій канали.
- `img.point(lut)` — метод для попиксельного застосування таблиці пошуку (Look-Up Table), що забезпечує миттєву зміну яскравості за наперед розрахованим масивом значень.
- `Image.merge()` — для синтезу фінального зображення зі змінених складових.
- `np.mean()` (з NumPy) або вбудована комбінація `sum() / len()` — для знаходження математичного сподівання (середньої інтенсивності) пікселів у каналах.

- `Image.new('L', ...)` — для створення допоміжних "чорних" масок при ізоляції окремих кольорних каналів.

Лістинг коду з коментарями:

```
def adjust_color_balance(
    input_path: str, output_path: str, r_shift: int = 0,
    g_shift: int = 0, b_shift: int = 0
) -> bool:
    """Коригує яскравість каналів R, G, B у діапазоні
    [-255..255]."""
    if not os.path.isfile(input_path):
        return False
    os.makedirs(os.path.dirname(output_path) or ".",
    exist_ok=True)
    with Image.open(input_path) as img:
        arr: np.ndarray = np.array(img.convert("RGB"),
        dtype=np.int16)
        arr[:, :, 0] += r_shift
        arr[:, :, 1] += g_shift
        arr[:, :, 2] += b_shift
        Image.fromarray(np.clip(arr, 0,
        255).astype(np.uint8)).save(output_path)
        print(f"Збережено: {output_path} (зсув
        R={r_shift:+d}, G={g_shift:+d}, B={b_shift:+d})")
        return True

def auto_color_balance(input_path: str, output_path: str,
    target: int = 128) -> bool:
    """Автоматичний баланс каналів за методичкою
    (вирівнювання до рівня 128)."""
    with Image.open(input_path) as img:
        arr: np.ndarray = np.array(img.convert("RGB"),
        dtype=np.float32)
        r_shift: int = int(round(target -
        float(np.mean(arr[:, :, 0]))))
        g_shift: int = int(round(target -
        float(np.mean(arr[:, :, 1]))))
        b_shift: int = int(round(target -
        float(np.mean(arr[:, :, 2]))))
        print(f"Зсуви каналів до {target}: R={r_shift:+d},
        G={g_shift:+d}, B={b_shift:+d}")
        return adjust_color_balance(input_path,
        output_path, r_shift, g_shift, b_shift)

def analyze_spectrum(input_path: str) -> None:
```



```

    """Аналіз спектру зображення та виявлення домінування
    каналів (стор. 30 методички)."""
    with Image.open(input_path) as img:
        arr: np.ndarray = np.array(img.convert("RGB"),
dtype=np.float64)
        means: list[float] = [float(np.mean(arr[:, :, i]))
for i in range(3)]
        total_energy: float = sum(means) or 1.0
        pcts: list[float] = [(m / total_energy) * 100 for m
in means]
        print(f"\nАналіз спектру:
{os.path.basename(input_path)}
({img.width}x{img.height})")
        print(f"    • Червоний (R): {means[0]:>5.1f} / 255
({pcts[0]:>4.1f}%)")
        print(f"    • Зелений (G): {means[1]:>5.1f} / 255
({pcts[1]:>4.1f}%)")
        print(f"    • Синій (B): {means[2]:>5.1f} / 255
({pcts[2]:>4.1f}%)")
        if max(pcts) - min(pcts) < 5.0:
            print("    ✓ Спектр збалансований (частки близькі
до 33.3%).")
        else:
            names: list[str] = ["червоний", "зелений",
"синій"]
            dom: list[str] = [names[i] for i in range(3) if
pcts[i] > 38.0]
            print(f"    ! Домінування: {'', '}.join(dom) if dom
else 'дисбаланс часток'}.")

def extract_channels(input_path: str, output_dir: str) ->
None:
    """Виділяє та зберігає канали RGB окремо (стор. 29-30
методички)."""
    os.makedirs(output_dir, exist_ok=True)
    with Image.open(input_path) as img:
        arr: np.ndarray = np.array(img.convert("RGB"))
        base: str =
os.path.splitext(os.path.basename(input_path))[0]
        print("\nЗбереження окремих каналів:")
        for idx, name in enumerate(("red", "green",
"blue")):
            ch: np.ndarray = np.zeros_like(arr)
            ch[:, :, idx] = arr[:, :, idx]
            out_path: str = os.path.join(output_dir,
f"{base}_{name}.png")

```

```
Image.fromarray(ch).save(out_path)
print(f"    • Канал {name.capitalize():<5}:
{out_path}")
```

Результат виконання програми:

```
=== Лабораторна робота №1: Комп'ютерна графіка ===
1. Конвертація форматів (з порівнянням розмірів)
2. Зміна розміру зображень (пропорційна, точна)
3. Перетворення кольорів (заміна цільового кольору)
4. Корекція колірного балансу та спектр
5. Вихід
Оберіть дію (1-5): 4
Шлях до зображення: test_images/test_landscape.jpg
1. Зсув яскравості  2. Баланс R,G,B  3. Авто-баланс  4. Спектр  5. Канали
Оберіть підпункт (1-5): 1
Шлях збереження: output_balance/brightness_plus30.jpg
Зсув яскравості [-255..255]: 30
Збережено: output_balance/brightness_plus30.jpg (зсув R=+30, G=+30, B=+30)
```

```
Оберіть підпункт (1-5): 2
Шлях збереження: output_balance/manual_rgb.jpg
Зсув R [-255..255]: -20
Зсув G [-255..255]: 10
Зсув B [-255..255]: 30
Збережено: output_balance/manual_rgb.jpg (зсув R=-20, G=+10, B=+30)
```

```
Оберіть підпункт (1-5): 3
Шлях збереження: output_balance/auto_balanced.jpg
Цільовий рівень (дефолт 128): 128
Зсуви каналів до 128: R=-68, G=+16, B=+56
Збережено: output_balance/auto_balanced.jpg (зсув R=-68, G=+16, B=+56)
```

```
Оберіть підпункт (1-5): 4
Аналіз спектру: test_landscape.jpg (1200x800)
• Червоний (R): 196.1 / 255 (51.6%)
• Зелений (G): 112.2 / 255 (29.5%)
• Синій (B): 71.8 / 255 (18.9%)
! Домінування: червоний.
```

```
Оберіть підпункт (1-5): 5
Директорія для каналів: output_channels

Збереження окремих каналів:
• Канал Red : output_channels\test_shapes_red.png
• Канал Green: output_channels\test_shapes_green.png
• Канал Blue : output_channels\test_shapes_blue.png
```

Висновок: У результаті виконання лабораторної роботи було повною мірою досягнуто поставленої мети. Під час виконання завдань було детально ознайомлено з основними принципами обробки цифрових зображень за допомогою програмного забезпечення, вивчено властивості популярних растрових форматів, алгоритмів їх стиснення та специфіку різних кольірних моделей. На основі здобутих знань успішно розроблено програму із використанням бібліотек Pillow та NumPy, яка виконує конвертацію форматів зображень із коректним обробленням прозорості, зміну їхніх розмірів, точкову заміну кольорів із врахуванням толерантності, а також комплексну корекцію кольорового балансу, включно з поканальним налаштуванням та автобалансом. Крім того, на практиці було досліджено, як саме перелічені операції впливають на фізичні характеристики файлів. Встановлено, що перехід до форматів зі стисненням із втратами (наприклад, JPEG) різко зменшує розмір файлу, але може негативно впливати на деталізацію. Водночас під час зміни розміру зображення збереження його візуальної якості та уникнення ефекту пікселізації критично залежить від вибору правильного математичного алгоритму інтерполяції (зокрема, бікубічної чи Lanczos) та збереження вихідних пропорцій. У підсумку було розвинуто та закріплено міцні практичні навички програмування і роботи зі спеціалізованими графічними бібліотеками, що дозволило ефективно автоматизувати процеси попиксельної обробки, маніпуляцій з кольірними каналами та загального математичного аналізу візуальних даних.

Відповіді на питання для самоконтролю (Лабораторна робота №1):

1. Що таке конвертація форматів зображень, і для чого вона потрібна?

Конвертація форматів — це процес перетворення цифрового зображення з одного стандарту збереження графічної інформації в інший. Вона потрібна для зменшення розміру файлу, збереження максимальної якості зображення, підтримки певних властивостей (наприклад, прозорості чи анімації) або для забезпечення сумісності з конкретними програмними середовищами чи пристроями відображення.

2. Які популярні формати зображень використовуються для збереження графічних даних, і які їх особливості?

- JPEG: Використовує стиснення з втратами. Має малий розмір, чудово підходить для фотографій, але не підтримує прозорість і втрачає якість при повторному збереженні.
- PNG: Застосовує стиснення без втрат. Підтримує альфа-канал (прозорість), ідеальний для логотипів та вебграфіки, але має більший розмір файлу.
- GIF: Підтримує анімацію і прозорість (лише 1 повністю прозорий колір), проте має обмеження до 256 кольорів.
- TIFF: Універсальний формат (із втратами чи без), зберігає максимальну якість і метадані, активно використовується у професійній поліграфії.
- BMP: Нестиснений формат, що зберігає вихідну якість "піксель-в-піксель", проте файли займають дуже багато місця.

3. Як змінюється розмір файлу зображення при конвертації між різними форматами? Розмір файлу суттєво змінюється залежно від алгоритмів стиснення цільового формату. Наприклад, якщо конвертувати великий нестиснений BMP у JPEG, розмір файлу різко зменшиться завдяки видаленню невидимих людському оку деталей (стиснення з втратами). Якщо конвертувати той самий BMP у PNG (стиснення без втрат), розмір також зменшиться, але файл буде значно більшим, ніж JPEG, оскільки вся інформація про якість залишається збереженою.

4. Що таке зміна розміру зображення, і які алгоритми використовуються для цього? Зміна розміру зображення — це процес перерахунку кількості пікселів у зображенні для його адаптації під нові габарити. Цей процес використовує методи інтерполяції:

- Найближчого сусіда: найпростіший і найшвидший метод, де новий піксель просто бере колір найближчого існуючого. Може створювати грубі "піксельні" артефакти.
- Лінійна інтерполяція: обчислює значення нового пікселя як середнє між сусідніми, роблячи перехід більш плавним.
- Бікубічна інтерполяція: найякісніший метод, що враховує не лише найближчі, а й віддаленіші сусідні пікселі, забезпечуючи плавні переходи та зберігаючи деталі без сильних спотворень.

5. Чому важливо зберігати співвідношення сторін під час зміни розміру зображення? Співвідношення сторін (пропорції ширини до висоти) необхідно зберігати для того, щоб не спотворювати геометричну форму об'єктів на зображенні. Якщо змінити ширину, не змінивши пропорційно висоту, зображення виглядатиме неприродно розтягнутим або стиснутим.

6. Як впливає зменшення розміру зображення на його якість і деталізацію? При зменшенні розміру частина пікселів назавжди видаляється (інформація втрачається). Відповідно, деталізація зображення погіршується. Якщо використати неякісні алгоритми інтерполяції, можуть виникати артефакти та ступінчастість ліній, тому для зменшення з максимальною якістю рекомендовано використовувати бікубічну інтерполяцію або Lanczos.

7. Що таке перетворення кольорів у зображенні, і які завдання воно може вирішувати? Перетворення кольорів — це адаптація кольорів зображення під певні технічні стандарти (наприклад, перехід від екранної моделі RGB до поліграфічної CMYK) або зміна відтінків задля художнього ефекту. Воно вирішує завдання точного відображення кольору на різних пристроях, виділення тіней/світла (за допомогою масок і кривих) та створення специфічного настрою чи тонування.

8. Як технічно реалізується заміна певного кольору на інший у зображенні? Технічно програма (чи графічний редактор) сканує матрицю пікселів зображення. Алгоритм порівнює числове значення кожного пікселя (його інтенсивність у каналах R, G, B чи інших моделях) із заданим цільовим кольором. Якщо значення співпадає (з урахуванням можливого діапазону похибки/допуску), інтенсивності каналів цього пікселя перезаписуються на нові значення цільового кольору.

9. Що таке колірний баланс, і які параметри його корекції використовуються в обробці зображень? Колірний баланс (баланс білого) — це налаштування співвідношення кольорів для досягнення природного і реалістичного вигляду. Основними параметрами його корекції є інтенсивність базових каналів (червоного, зеленого та синього) та зсув відтінків для усунення перекосу в "теплі" (жовто-червоні) або "холодні" (синьо-блакитні) тони, які могли з'явитися через некоректне освітлення під час зйомки.

10. Як впливає корекція колірного балансу на сприйняття зображення користувачем? Завдяки правильній корекції кольори стають більш природними (наприклад, білий аркуш паперу виглядає білим, а не синюватим). Це покращує реалістичність. Крім того, цілеспрямований зсув балансу (колірне тонування) дозволяє створювати бажану атмосферу та емоційний настрій фотографії (затишок і тепло від додавання жовтих тонів або ж холод і строгість від синіх відтінків).